

Social Network Explorer (SNE)

Final Capstone Report

School	School of Engineering & Technology
Program	B.Sc (H) CS / DS / Cyber Security
Course	Basics of Data Structures
Code	ETCCBD201
Coverage	Unit 1 + Unit 2 + Unit 3 + Unit 4 (Full Syllabus)
Type	Industry-Oriented Mini System (CLI Application)

Python 3.10+ | No External Dependencies | GitHub-Ready

1. Design Overview

The Social Network Explorer (SNE) is a modular, command-line Python application that simulates core features of real social platforms such as LinkedIn and Instagram. It is organised into five independent modules to enforce separation of concerns — a practice standard in industry codebases.

Module	Responsibility	Key DS
profiles.py	User profile storage & retrieval	HashTable (dict)
network_graph.py	Friendship / follow network	Adjacency List (Graph)
algorithms.py	BFS, DFS, friend suggestion	Queue, Stack, Set
sorting.py	Ranking algorithms	Array (in-place / auxiliary)
main.py	CLI menu + demo mode (app entry point)	—

All modules share a single **HashTable** instance (profiles.py) and a single **SocialGraph** instance (network_graph.py). This avoids data duplication and ensures that validation (e.g., checking whether a user exists before adding a friendship) always queries the authoritative store.

2. Data Structures & Implementation

2.1 Hash Table — User Profiles

User profiles are stored in a **HashTable** backed by Python's built-in dict, which uses an open-addressing scheme with pseudo-random probing and dynamic resizing at a load factor of ~ 0.7 . The string key (user_id) is hashed via Python's hash() built-in to determine the bucket index.

- add_user — $O(1)$ amortised
- get_user_profile — $O(1)$ amortised
- update_user_profile — $O(1)$ amortised
- list_all_users — $O(n)$

2.2 Graph — Adjacency List (Friendship Network)

The social graph is represented as a dictionary of lists: `adj[user_id] = [neighbour_ids]`. This adjacency-list format is preferred over an adjacency matrix for sparse social graphs because it uses $O(V + E)$ space versus $O(V^2)$ for a matrix.

- add_friendship — $O(1)$
- remove_friendship — $O(\text{degree})$ linear scan
- get_friends — $O(1)$ (returns existing list)
- Space complexity — $O(V + E)$

2.3 Queue — BFS

`collections.deque` is used as the BFS queue, giving $O(1)$ amortised append and `popleft` — critical for efficient traversal. Each queue entry is a (current_node, path_so_far) tuple so the full path is reconstructed

without a separate parent-pointer dict.

2.4 Stack — DFS (Iterative)

DFS uses an explicit Python list as a stack (`stack.pop()` = LIFO). This avoids Python's default recursion limit (1000 frames) and makes depth control straightforward: each stack entry carries a depth counter, and neighbours are only pushed when `depth < max_depth`.

3. Algorithm Analysis

3.1 BFS — Shortest Path (Degrees of Separation)

BFS explores the graph level by level, guaranteeing the shortest path in an unweighted graph. The visited set prevents re-visiting nodes, ensuring termination even in cycles.

```
bfs_shortest_path(adj, 'u1', 'u8', store) → ['u1', 'u3', 'u5', 'u8'] (3 degrees of separation)
```

Time: $O(V + E)$ — every vertex and edge is visited at most once.

Space: $O(V)$ — visited set and queue hold at most V entries.

3.2 DFS — Friends-of-Friends (Depth-Limited)

Iterative DFS discovers all users reachable from a start node within k hops. The depth counter prevents traversal beyond the requested depth, and the visited set avoids infinite loops in cyclic graphs.

```
dfs_friends_of_friends(adj, 'u1', depth=2, store) → ['u2', 'u3', 'u5', 'u6', 'u4']  
(5 users at depth  $\leq 2$ )
```

Time: $O(V + E)$ bounded to the reachable sub-graph within `max_depth`.

Space: $O(V)$ — visited set and stack.

3.3 Friend Suggestion — Common-Interest Scoring

For a given user U , all non-connected users are scored by `len(set(U.interests) & set(candidate.interests))`. Set intersection runs in $O(\min(|A|, |B|))$ using Python's hash set. Candidates are then sorted by score in descending order using **Merge Sort**.

```
get_friend_suggestions('u1', adj, store, top_n=5) → Eve [score=3] AI, music, travel  
Frank[score=2] AI, tech Grace[score=2] music, travel
```

4. Sorting Algorithms — Unit 3

Two sorting algorithms are implemented from scratch in **sorting.py**. Neither uses Python's built-in `sorted()` or `.sort()`. Both accept a key callable and a reverse flag, matching the interface of the standard library for easy comparison.

Algorithm	Best	Average	Worst	Space	Stable?
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes

Why Insertion Sort for small n ?

Insertion Sort has lower constant factors and zero recursion overhead. In the demo run (8 elements) Insertion Sort was faster despite its $O(n^2)$ worst case — this is consistent with CPython benchmarks showing Insertion Sort outperforms Merge Sort for $n < \sim 20$.

Why Merge Sort for friend ranking?

Merge Sort guarantees $O(n \log n)$ regardless of input order. For production systems with thousands of candidate users, consistent latency matters more than best-case speed. Merge Sort is also stable — equal-scored candidates retain their original order, which is desirable for ranking fairness.

5. Conceptual — Recommendation System

This section addresses the mandatory conceptual task from the assignment.

5.1 Hashing Users by Interest

A secondary hash map `interest_index: dict[str, list[str]]` maps each interest string to the list of user IDs who share it. Building this index costs $O(n \cdot I)$ time where n is the number of users and I is the average number of interests per user. Querying it for a given interest costs $O(1)$ average — hash lookup — yielding the candidate list instantly without scanning all users.

5.2 Computing Common Interests

For a target user T and a candidate C , the common-interest score is computed as $\text{len}(\text{set}(T.\text{interests}) \cap \text{set}(C.\text{interests}))$. Python's set intersection uses a hash-based approach: $O(\min(|T|, |C|))$ time. Across U candidate users the total scoring cost is $O(U \cdot I)$.

5.3 Sorting Candidates by Score

After scoring, the candidate list is sorted in descending order by score using Merge Sort ($O(n \log n)$). The top- k results are sliced from the front of the sorted list in $O(k)$. An alternative for very large candidate sets is a **min-heap of size k** ($O(n \log k)$), but Merge Sort was chosen here to satisfy the Unit-3 requirement to implement and use a sorting algorithm directly.

5.4 Bonus — Spatial Search (Coordinates)

If user profiles include (latitude, longitude) coordinates, the best Unit-4 structure for proximity search is a **K-D Tree** (k-dimensional tree). A K-D Tree partitions 2-D space recursively, giving $O(\log n)$ average nearest-neighbour search versus $O(n)$ for a brute-force scan. A **Quadtree** is an alternative that partitions space into four quadrants at each level and works better when data is clustered geographically. Both outperform flat lists for spatial queries at scale.

6. Validation & Error Handling

Every public function validates its inputs and prints a descriptive [ERROR] message rather than raising an uncaught exception. The following edge cases are handled:

- Duplicate user_id — `add_user` returns False with an error message.
- Unknown user_id — `get/update/delete` print [ERROR] and return None/False.
- Self-friendship — `add_friendship` rejects `user1 == user2`.
- Friendship with non-existent user — both IDs are validated before any edge is added.
- Duplicate friendship — checked with `has_connection` before appending.
- BFS / DFS with no path — returns [] with a clear 'No connection found' message.
- Empty interest list — handled gracefully; score = 0, still appears in suggestions.
- Invalid age input in CLI — `prompt_int` returns a default rather than crashing.

7. Sample CLI Output

The following is a partial extract from **output.txt** (demo mode):

```
[OK] User 'u1' (Alice) added successfully. [OK] User 'u3' (Carol) added
successfully. [OK] 'u1' is now friends with 'u3'. [OK] Connection between 'u4' and
'u7' removed. BFS: Shortest path from 'Alice' -> 'Heidi' Path : Alice(u1) ->
Carol(u3) -> Eve(u5) -> Heidi(u8) Degrees: 3 degree(s) of separation DFS:
Friends-of-friends of 'Alice' up to depth 2 1. Bob (u2) 2. Carol (u3) 3. Eve (u5) 4.
Frank (u6) 5. Dave (u4) Total reachable: 5 user(s) Friend Suggestions for 'Alice'
(u1): # Name ID Score Common Interests 1 Eve u5 3 AI, music, travel 2 Frank u6 2 AI,
tech 3 Grace u7 2 music, travel
```

8. Complexity Summary

Operation	Time (avg)	Space	Module
add_user	$O(1)$	$O(1)$	profiles.py
get_user_profile	$O(1)$	$O(1)$	profiles.py
update_user_profile	$O(1)$	$O(1)$	profiles.py
add_friendship	$O(1)$	$O(1)$	network_graph.py
remove_friendship	$O(\text{degree})$	$O(1)$	network_graph.py
get_friends	$O(1)$	$O(1)$	network_graph.py
BFS shortest path	$O(V + E)$	$O(V)$	algorithms.py
DFS depth search	$O(V + E)$	$O(V)$	algorithms.py
Friend suggestion score	$O(U * I)$	$O(U)$	algorithms.py
Insertion Sort	$O(n^2)$	$O(1)$	sorting.py
Merge Sort	$O(n \log n)$	$O(n)$	sorting.py

9. References

- [1] Cormen, T. H. et al. — Introduction to Algorithms (4th ed.), MIT Press, 2022.
- [2] Python Software Foundation — collections.deque (Python 3 docs), docs.python.org.
- [3] Python Software Foundation — Built-in Types: dict, docs.python.org.
- [4] Sedgewick, R. & Wayne, K. — Algorithms (4th ed.), Addison-Wesley, 2011.
- [5] Skiena, S. — The Algorithm Design Manual (3rd ed.), Springer, 2020.

End of Report